(continued from previous page)

```
collected 1 item

test_assert1.py F                                                    [100%]

================================= FAILURES =================================
_____________________________ test_function ______________________________

    def test_function():
>       assert f() == 4
E       assert 3 == 4
E        +  where 3 = f()

test_assert1.py:6: AssertionError
========================== short test summary info ==========================
FAILED test_assert1.py::test_function - assert 3 == 4
============================== 1 failed in 0.12s ===============================
```

`pytest` has support for showing the values of the most common subexpressions including calls, attributes, comparisons, and binary and unary operators. (See *Demo of Python failure reports with pytest*). This allows you to use the idiomatic python constructs without boilerplate code while not losing introspection information.

If a message is specified with the assertion like this:

```
assert a % 2 == 0, "value was odd, should be even"
```

it is printed alongside the assertion introspection in the traceback.

See *Assertion introspection details* for more information on assertion introspection.

2.2.2 Assertions about approximate equality

When comparing floating point values (or arrays of floats), small rounding errors are common. Instead of using `assert abs(a - b) < tol` or `numpy.isclose`, you can use *`pytest.approx()`*:

```
import pytest
import numpy as np


def test_floats():
    assert (0.1 + 0.2) == pytest.approx(0.3)


def test_arrays():
    a = np.array([1.0, 2.0, 3.0])
    b = np.array([0.9999, 2.0001, 3.0])
    assert a == pytest.approx(b)
```

`pytest.approx` works with scalars, lists, dictionaries, and NumPy arrays. It also supports comparisons involving NaNs.

See *`pytest.approx()`* for details.

(continued from previous page)

```
# contents of test_image.py
def test_histogram(image_file):
    img = load_image(image_file)
    # compute and test histogram
```

See *tmp_path_factory API* for details.

2.6.3 The `tmpdir` and `tmpdir_factory` fixtures

The `tmpdir` and `tmpdir_factory` fixtures are similar to `tmp_path` and `tmp_path_factory`, but use/return legacy py.path.local objects rather than standard `pathlib.Path` objects.

Note

These days, it is preferred to use `tmp_path` and `tmp_path_factory`.

In order to help modernize old code bases, one can run pytest with the legacypath plugin disabled:

```
pytest -p no:legacypath
```

This will trigger errors on tests using the legacy paths. It can also be permanently set as part of the *addopts* parameter in the config file.

See *tmpdir tmpdir_factory* API for details.

2.6.4 Temporary directory location and retention

The temporary directories, as returned by the *tmp_path* and (now deprecated) *tmpdir* fixtures, are automatically created under a base temporary directory, in a structure that depends on the *--basetemp* option:

- By default (when the *--basetemp* option is not set), the temporary directories will follow this template:

  ```
  {temproot}/pytest-of-{user}/pytest-{num}/{testname}/
  ```

  where:

  - `{temproot}` is the system temporary directory as determined by `tempfile.gettempdir()`. It can be overridden by the *PYTEST_DEBUG_TEMPROOT* environment variable.
  - `{user}` is the user name running the tests,
  - `{num}` is a number that is incremented with each test suite run
  - `{testname}` is a sanitized version of *the name of the current test*.

  The auto-incrementing `{num}` placeholder provides a basic retention feature and avoids that existing results of previous test runs are blindly removed. By default, the last 3 temporary directories are kept, but this behavior can be configured with *tmp_path_retention_count* and *tmp_path_retention_policy*.

- When the *--basetemp* option is used (e.g. `pytest --basetemp=mydir`), it will be used directly as base temporary directory:

  ```
  {basetemp}/{testname}/
  ```

  Note that there is no retention feature in this case: only the results of the most recent run will be kept.

> **Warning**
>
> The directory given to *--basetemp* will be cleared blindly before each test run, so make sure to use a directory for that purpose only.

When distributing tests on the local machine using `pytest-xdist`, care is taken to automatically configure a `basetemp` directory for the sub processes such that all temporary data lands below a single per-test run temporary directory.

2.7 How to monkeypatch/mock modules and environments

Sometimes tests need to invoke functionality which depends on global settings or which invokes code which cannot be easily tested such as network access. The `monkeypatch` fixture helps you to safely set/delete an attribute, dictionary item or environment variable, or to modify `sys.path` for importing.

The `monkeypatch` fixture provides these helper methods for safely patching and mocking functionality in tests:

- *monkeypatch.setattr(obj, name, value, raising=True)*
- *monkeypatch.delattr(obj, name, raising=True)*
- *monkeypatch.setitem(mapping, name, value)*
- *monkeypatch.delitem(obj, name, raising=True)*
- *monkeypatch.setenv(name, value, prepend=None)*
- *monkeypatch.delenv(name, raising=True)*
- *monkeypatch.syspath_prepend(path)*
- *monkeypatch.chdir(path)*
- *monkeypatch.context()*

All modifications will be undone after the requesting test function or fixture has finished. The `raising` parameter determines if a `KeyError` or `AttributeError` will be raised if the target of the set/deletion operation does not exist.

Consider the following scenarios:

1. Modifying the behavior of a function or the property of a class for a test e.g. there is an API call or database connection you will not make for a test but you know what the expected output should be. Use *monkeypatch.setattr* to patch the function or property with your desired testing behavior. This can include your own functions. Use *monkeypatch.delattr* to remove the function or property for the test.

2. Modifying the values of dictionaries e.g. you have a global configuration that you want to modify for certain test cases. Use *monkeypatch.setitem* to patch the dictionary for the test. *monkeypatch.delitem* can be used to remove items.

3. Modifying environment variables for a test e.g. to test program behavior if an environment variable is missing, or to set multiple values to a known variable. *monkeypatch.setenv* and *monkeypatch.delenv* can be used for these patches.

4. Use `monkeypatch.setenv("PATH", value, prepend=os.pathsep)` to modify `$PATH`, and *monkeypatch.chdir* to change the context of the current working directory during a test.

5. Use *monkeypatch.syspath_prepend* to modify `sys.path` which will also call `pkg_resources.fixup_namespace_packages` and `importlib.invalidate_caches()`.

6. Use *monkeypatch.context* to apply patches only in a specific scope, which can help control teardown of complex fixtures or patches to the stdlib.

See the monkeypatch blog post for some introduction material and a discussion of its motivation.

2.11.4 Accessing captured output from a test function

The *capsys*, *capteesys*, *capsysbinary*, *capfd*, and *capfdbinary* fixtures allow access to `stdout`/`stderr` output created during test execution.

Here is an example test function that performs some output related checks:

```
def test_myoutput(capsys):  # or use "capfd" for fd-level
    print("hello")
    sys.stderr.write("world\n")
    captured = capsys.readouterr()
    assert captured.out == "hello\n"
    assert captured.err == "world\n"
    print("next")
    captured = capsys.readouterr()
    assert captured.out == "next\n"
```

The `readouterr()` call snapshots the output so far - and capturing will be continued. After the test function finishes the original streams will be restored. Using *capsys* this way frees your test from having to care about setting/resetting output streams and also interacts well with pytest's own per-test capturing.

The return value of `readouterr()` is a `namedtuple` with two attributes, `out` and `err`.

If the code under test writes non-textual data (`bytes`), you can capture this using the *capsysbinary* fixture which instead returns `bytes` from the `readouterr` method.

If you want to capture at the file descriptor level you can use the *capfd* fixture which offers the exact same interface but allows to also capture output from libraries or subprocesses that directly write to operating system level output streams (FD1 and FD2). Similarly to *capsysbinary*, *capfdbinary* can be used to capture `bytes` at the file descriptor level.

To temporarily disable capture within a test, the capture fixtures have a `disabled()` method that can be used as a context manager, disabling capture inside the `with` block:

```
def test_disabling_capturing(capsys):
    print("this output is captured")
    with capsys.disabled():
        print("output not captured, going directly to sys.stdout")
    print("this output is also captured")
```

Note

When a capture fixture such as *capsys* or *capfd* is used, it takes precedence over the global capturing configuration set via command-line options such as `-s` or `--capture=no`.

This means that output produced within a test using a capture fixture will still be captured and available via `readouterr()`, even if global capturing is disabled.

2.12 How to capture warnings

Starting from version `3.1`, pytest now automatically catches warnings during test execution and displays them at the end of the session:

```
# content of test_show_warnings.py
import warnings
```

(continues on next page)

(continued from previous page)

```
>>> with warns(UserWarning, match=r"must be \d+$"):
...     warnings.warn("this is not here", UserWarning)
...
Traceback (most recent call last):
  ...
Failed: Regex pattern did not match any of the 1 warnings emitted.
 Regex: ...
 Emitted warnings: ...UserWarning...

>>> with warns(UserWarning, match=re.escape("issue with foo() func")):
...     warnings.warn("issue with foo() func")
...
```

The function also returns a list of all raised warnings (as `warnings.WarningMessage` objects), which you can query for additional information:

```
with pytest.warns(RuntimeWarning) as record:
    warnings.warn("another warning", RuntimeWarning)

# check that only one warning was raised
assert len(record) == 1
# check that the message matches
assert record[0].message.args[0] == "another warning"
```

Alternatively, you can examine raised warnings in detail using the *recwarn* fixture (see *below*).

The *recwarn* fixture automatically ensures to reset the warnings filter at the end of the test, so no global state is leaked.

2.12.9 Recording warnings

You can record raised warnings either using the *pytest.warns()* context manager or with the *recwarn* fixture.

To record with *pytest.warns()* without asserting anything about the warnings, pass no arguments as the expected warning type and it will default to a generic Warning:

```
with pytest.warns() as record:
    warnings.warn("user", UserWarning)
    warnings.warn("runtime", RuntimeWarning)

assert len(record) == 2
assert str(record[0].message) == "user"
assert str(record[1].message) == "runtime"
```

The *recwarn* fixture will record warnings for the whole function:

```
import warnings


def test_hello(recwarn):
    warnings.warn("hello", UserWarning)
    assert len(recwarn) == 1
    w = recwarn.pop(UserWarning)
    assert issubclass(w.category, UserWarning)
```

(continues on next page)